

Table 5: Common pitfalls and how to avoid them

Common pitfall	Description	How to avoid it
Using large base images	Large images make your container bulky and slow to download.	Use lightweight images like openjdk:17-jdk-alpine and multi-stage builds.
Ignoring <i>.dockerignore</i>	Including unnecessary files bloats your build context and image size.	Create a <i>.dockerignore</i> file to exclude irrelevant files.
Hardcoding configuration inside image	Embedding environment-specific settings reduces flexibility and forces rebuilds.	Use environment variables or external config files.
Running containers as root	Running as root poses security risks if the container is compromised.	Create and run as a non-root user in the Dockerfile.
Over-exposing ports	Exposing unnecessary ports increases the attack surface.	Expose only the ports your app needs.
Forgetting health checks	Without health checks, unhealthy containers may not be detected or restarted.	Add Docker health checks for automatic monitoring.
Logging to files inside container	Writing logs to container files complicates log management and can cause storage issues.	Log to standard output (console) instead.
Neglecting image updates	Using outdated base images and dependencies can cause security vulnerabilities.	Regularly update base images and rebuild containers.

up less disk space, downloads quicker, and is more secure. Think of it like baking a cake in one kitchen and then only packing the finished cake (leaving behind the baking tools and ingredients) into a box for delivery.

Here's an example of a multi-stage Dockerfile for a Spring Boot Java app.

Stage 1: Build the app using Maven:

```
FROM maven:3.8.6-openjdk-17 AS build
```

Set working directory inside the builder container:

```
WORKDIR /build
```

Copy source code and *pom.xml* files into the container:

```
COPY pom.xml . COPY src ./src
```

Build the project and package it as a jar:

```
RUN mvn clean package -DskipTests
```

Stage 2: Create a lightweight image for running the app:

```
FROM openjdk:17-jdk-alpine
```

Set working directory inside the runtime container:

```
WORKDIR /app
```

Copy the JAR built in the first stage into this stage:

```
COPY --from=build /build/target/myapp.jar ./myapp.jar
```

Run the Java app when the container starts:

```
CMD ["java", "-jar", "myapp.jar"]
```

This is how it works:

- The first stage (build) uses a Maven image with JDK to compile your app and create the JAR.
- The second stage starts from a small Java runtime image (openjdk:17-jdk-alpine) and copies only the built JAR from the first stage.
- This way, the final image doesn't include Maven, source code, or build files — only the essential runtime and your app.

Docker has truly transformed the way we build, ship, and deploy Java applications by offering a consistent and portable environment. When you containerise your Java apps, you can be confident they'll run smoothly across various machines and cloud platforms. By sticking to best practices like using lightweight images, multi-stage builds, and effective configuration management, you can create Docker containers that are not only efficient but also secure and easy to maintain. As you progress on your Docker journey, diving into advanced topics like Docker Compose for managing multi-container applications, Kubernetes for orchestration, and integrating Docker with CI/CD pipelines will take your development workflow to the next level. **END** 🐧

 By: Megha Kumari Agarwal

The author works at a service-based multinational company (MNC) in Gurugram, where she actively contributes to major IT projects.